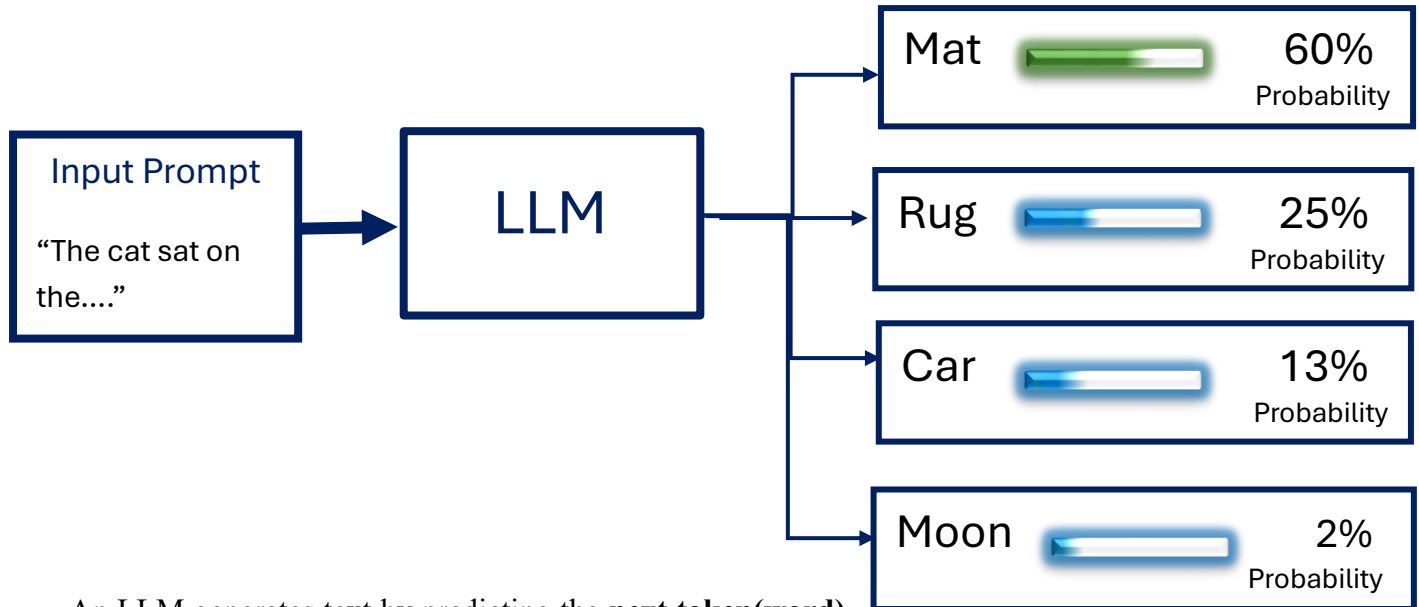


LLM Inference

❖ What are LLMs.

LLMs are Next TOKEN (Word) Predictor



An LLM generates text by predicting the **next token(word)** based on the tokens that have already been provided. In the diagram, the input prompt is:

“The cat sat on the...”

The LLM processes this input and calculates scores for many possible next tokens based on its learning dataset. These scores are converted into probabilities, representing how likely each token is to be selected as the next token.

For example, the model may assign:

- **Mat → 60% probability**
- **Rug → 25% probability**
- **Car → 13% probability**
- **Moon → 2% probability**

The token with the highest probability is **“Mat”**, so the next token based on the highest probability can be selected **“Mat”** as the next token.

1. Training vs Inference

Imagine we want to create an LLM.

1.1 What is Training?

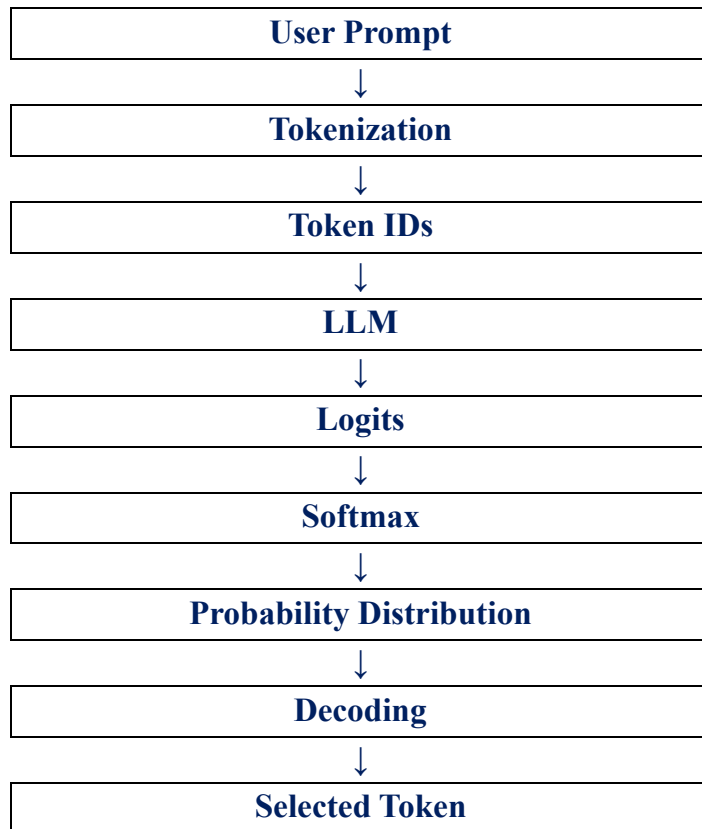
Training means teaching a model using a large amount of data. We give the dataset to the model, and the model learns patterns from that data. Based on what it learns, the model knows how to respond to different inputs.

The model after this learning process is called a **trained model**.

1.2 What is LLM Inference

LLM inference is the process of using a trained model to produce an output from new input.

2. How Does an LLM Predict the Next Token?



2.1 Tokenization

Before an LLM can process text, the input text is converted into smaller pieces called tokens. These tokens can be complete words, parts of words, punctuation marks, or other text fragments.

Example:

Input: "The cat sat on the"

|
"The" " cat" " sat"

Then these tokens are converted into token ids. These ids are based the tokenizer's vocabulary.

"The" → 183

" cat" → 742

" sat" → 981

" on" → 192

" the" → 183

2.2 Logits

After the input text is converted into token IDs, these tokens are further send to the LLM. The model produces a numerical score for every possible next token. These raw scores are called **logits**. Logits indicate how strongly the model matches the token to the pervious token. However, logits are not probabilities. They can be positive or negative and do not have to add up to any particular value.

Example:

Suppose the input is: **“The capital of France is”**

The model may produce the following logits:

Token	Logit
Paris	8
London	3
Berlin	2
Mumbai	1

Here, **Paris has the highest logit**, meaning the model gives it the highest raw score among these candidates.

However, we cannot say that Paris has an **8% or 80% probability** just because its logit is 8.

The logits need to be converted into probabilities before we can consider them as probabilities.

2.3 Softmax — Converting Logits into Probabilities

Softmax is a mathematical function used to convert the raw logits produced by an LLM into a probability distribution.

Formula

Where:

- $P(i)$ = probability of a token
- z_i = logit of that token
- e = Euler's number, approximately **2.718**
- $\sum e^{z_j}$ = sum of the exponentials of all logits

$$P(i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Example: Converting Logits into Probabilities

Input: "The capital of France is"

Token	Logit
Paris	8
London	3
Berlin	2
Mumbai	1

Step 1: Calculate e^{logit}

For each token:

$$e^8 = 2980.958$$

$$e^3 = 20.086$$

$$e^2 = 7.389$$

$$e^1 = 2.718$$

So:

Token	Logit	e^{logit}
Paris	8	2980.958
London	3	20.086
Berlin	2	7.389
Mumbai	1	2.718

Step 2: Calculate the denominator

Add all the exponential values:

$$2980.958 + 20.086 + 7.389 + 2.718 \\ = 3011.151$$

Step 3: Calculate probability of each token

For **Paris**:

$$P(\text{Paris}) = \frac{2980.958}{3011.151}$$

Therefore:

$$P(\text{Paris}) \approx 0.98998$$

Similarly:

$$\text{Paris} = 98.998\%$$

$$\text{London} = 0.667\%$$

$$\text{Berlin} = 0.245\%$$

$$\text{Mumbai} = 0.090\%$$

Note: Softmax does not select the next token. It only converts the logits into probabilities.

2.4 Decoding

Decoding is the process of selecting the next token from the probability generated by the LLM.

1. Greedy Decoding

Greedy decoding selects the token with the highest probability.

Eg. Paris = 98.998% $\longrightarrow$ Selected
London = 0.667%
Berlin = 0.245%
Mumbai = 0.090%

2. Sampling

In sampling, the model selects a token based on the probability distribution rather than always selecting the highest-probability token. This allows lower-probability tokens to sometimes be selected, which can produce more varied responses.

But what will decide that token can be selected of lower probability also.

2.1 Temperature

- Temperature is a decoding parameter that controls how random or predictable the token selection is during LLM inference.
- Temperature is applied to the logits before they are converted into probabilities using Softmax.
- A lower temperature makes the probability centric to highest-probability tokens, making the output more predictable. A higher temperature increases the chance of selecting lower-probability tokens.

$$P(i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

Formula

Where: Z^i = Logit of the token

T = Temperature

$P(i)$ = Probability

Example: Effect of Temperature

Use the logits you already have:

Token	Logit
Paris	8
London	3
Berlin	2
Mumbai	1

a. Temperature = 1

When: $T = 1$

the logits remain unchanged:

Paris → 8

London → 3

Berlin → 2

Mumbai → 1

The probabilities are the same as your Softmax example:

Paris → 98.998%

London → 0.667%

Berlin → 0.245%

Mumbai → 0.090%

So, Paris is overwhelmingly likely to be selected.

b. Temperature = 0.5

Now: $T = 0.5$

We divide every logit by 0.5:

Paris: $\frac{8}{0.5} = 16$

London: 6

Berlin: 4

Mumbai: 2

Probabilities:

Paris → 99.995%

London → 0.0045%

Berlin → 0.0006%

Mumbai → 0.00008%

The probability of Paris became even higher.

Lower temperature → Sharper distribution → More predictable output

c. Temperature = 2

Divide every logit by 2:

New logits:

Paris → 4

London → 1.5

Berlin → 1

Mumbai → 0.5

After Softmax:

Paris → 86.05%

London → 7.06%

Berlin → 4.28%

Mumbai → 2.60%

Now Paris is still the most likely token, but the other tokens have a **much higher probability** than before.

2.2 Top-K Sampling

Top-K sampling is a decoding technique that limits the number of candidate tokens considered for the next-token selection.

The value **K** represents the number of highest-probability tokens that will remain as candidates.

For example, suppose the LLM produces this probability distribution:

Token	Probability
Paris	40%
London	25%
Berlin	15%
Rome	10%
Tokyo	5%
Delhi	3%

The total probability is: $40+25+15+10+5+3+2=100\%$

Suppose **K = 3**,

the model keeps only the **3 highest-probability tokens**:

Paris → 40%

London → 25%

Berlin → 15%

The remaining tokens are removed from the candidate set:

Rome

Tokyo

Delhi

What happens next?

The remaining probabilities are **renormalized** so that they add up to 100%.

The remaining probability mass is:

$$40+25+15=80\%$$

Now calculate the new probabilities.

For Paris:

$$P(\text{Paris}) = \frac{40}{80} = 50\%$$

Similarly:

$$P(\text{London}) = \frac{25}{80} = 31.25\%$$

$$P(\text{Berlin}) = \frac{15}{80} = 18.75\%$$

So, Top K is used to reduce the number of tokens that are have highest probability.

2.3 Top-P / Nucleus Sampling

Top-P sampling selects the most likely tokens until their total probability reaches the given P value.

Example: Use the same distribution:

Token	Probability
Paris	40%
London	25%
Berlin	15%
Rome	10%
Tokyo	5%

Suppose: Top-P = 80%

Now add probabilities from highest to lowest: Paris : 40%

Cumulative: 40%

Add London: $40+25=65\%$

Add Berlin: $40+25+15=80\%$

3. Final Token Selection

After applying Top-K or Top-P, the model has a smaller list of tokens to choose from. The model does not always select the first or highest-probability token. It chooses a token based on the probabilities. The model's learning also affects these probabilities.

Temperature controls how predictable or random the selection is:

- Lower temperature → more predictable output.
- Higher temperature → more random and varied output

4. Practical Demonstration

4.1 Experiment Setup

The experiment was implemented in a Jupyter Notebook using Python and the Open Router API.

Tools used:

- Python
- Jupyter Notebook
- OpenRouter API
- Pandas

The notebook contains the complete implementation, different parameter values, generated outputs, and observations.

GitHub: You can check the link for the implementation [LLM Inference](#)